Calculatorz

Arithmetic Expression Evaluator in C++

Test Cases for Tokenizer Module

Version 1.0

Revision History

Date	Version	Description	Author
2026-05-04	1.0	Add 10 test cases for the tokenizer module	Jason (Quy)
2026-05-06	1.1	Final polishing	Isaiah

Table of Contents

Test Cases

1. Purpose

This *Test Cases* specification document for the Arithmetic Expression Evaluator in C++ defines the test cases for the `tokenizer` module.

2. Basic Valid Expression

2.1 Test case identifier

`TC-TOK-01`

2.2 Test item

Tokenizer module: This test checks whether the tokenizer correctly identifies parentheses, numbers, addition, and division operators in a normal arithmetic expression.

2.3 Input specifications

Input: (3 + 5) / 6

2.4 Output specifications

Output: <{LeftParen "("}, {Number "3"}, {Plus "+"}, {Number "5"}, {RightParen ")"}, {Divide "/"}, {Number "6"}>

2.5 Environmental needs

Standard execution environment.

3. Whitespace Handling

3.1 Test case identifier

`TC-TOK-02`

3.2 Test item

Tokenizer module: This test checks whether the tokenizer ignores extra spaces and still produces the correct tokens.

3.3 Input specifications

Input: 10 * 2

3.4 Output specifications

Output: <{Number "10"}, {Multiply "*"}, {Number "2"}>

3.5 Environmental needs

Standard execution environment.

4. Multi-digit Numbers

4.1 Test case identifier

`TC-TOK-03`

4.2 Test item

Tokenizer module: This test checks whether the tokenizer groups multiple digits into one number token instead of separating each digit.

4.3 Input specifications

Input: 123 + 456

4.4 Output specifications

Output: <{Number "123"}, {Plus "+"}, {Number "456"}>

4.5 Environmental needs

Standard execution environment.

5. Decimal Numbers

5.1 Test case identifier

`TC-TOK-04`

5.2 Test item

Tokenizer module: This test checks whether the tokenizer correctly recognizes floating-point numbers.

5.3 Input specifications

Input: 3.14 * 2.5

5.4 Output specifications

Output: <{Number "3.14"}, {Multiply "*"}, {Number "2.5"}>

5.5 Environmental needs

Standard execution environment.

6. Invalid Character

6.1 Test case identifier

`TC-TOK-05`

6.2 Test item

Tokenizer module: This test checks whether the tokenizer detects unsupported characters that are not numbers, operators, or parentheses.

6.3 Input specifications

Input: 5 + a

6.4 Output specifications

Output: Error: Invalid token "a"

6.5 Environmental needs

Standard execution environment.

7. Unary Minus (Negative Numbers)

7.1 Test case identifier

`TC-TOK-06`

7.2 Test item

Tokenizer module: This test checks whether the tokenizer correctly handles a minus sign used as part of a negative number.

7.3 Input specifications

Input: -5 + 3

7.4 Output specifications

Output: <{Number "-5"}, {Plus "+"}, {Number "3"}>

7.5 Environmental needs

Standard execution environment.

8. Empty Input

8.1 Test case identifier

`TC-TOK-07`

8.2 Test item

Tokenizer module: This test checks whether the tokenizer handles empty input without crashing.

8.3 Input specifications

Input: “”

8.4 Output specifications

Output: <>

8.5 Environmental needs

Standard execution environment.

9. Malformed Number

9.1 Test case identifier

`TC-TOK-08`

9.2 Test item

Tokenizer module: This test checks whether the tokenizer detects invalid number formats, such as a number with multiple decimal points.

9.3 Input specifications

Input: 3.1.4 + 2

9.4 Output specifications

Output: Error: Invalid number "3.1.4"

9.5 Environmental needs

Standard execution environment.

10. Consecutive Operators

10.1 Test case identifier

`TC-TOK-09`

10.2 Test item

Tokenizer module: This test checks whether the tokenizer or validation stage detects invalid consecutive operators.

10.3 Input specifications

Input: 5 ++ 3

10.4 Output specifications

Output: Syntax error at position 3: unexpected operator placement

10.5 Environmental needs

Standard execution environment.

11. Parentheses Tokenization (No Validation)

11.1 Test case identifier

`TC-TOK-10`

11.2 Test item

Tokenizer module: This test checks whether the tokenizer correctly creates tokens for parentheses without checking whether the parentheses are balanced.

11.3 Input specifications

Input: (2 + 3

11.4 Output specifications

Output: <{LeftParen "("}, {Number "2"}, {Plus "+"}, {Number "3"}>

11.5 Environmental needs

Standard execution environment.